ipst
cnam

Analyse du besoin et modélisation du projet

Projet NSY103 « Mon D.O.S. »

Groupe de trois étudiants :

Jade BEAUJOUR
Nesrine BENHALIMA
Nael FARES

1. Notre compréhension du besoin

Nous comprenons le projet comme la réalisation d'un mini système d'exploitation permettant à l'utilisateur de manipuler un système de fichiers à partir de commandes saisies dans un premier terminal. Nous voulons que l'utilisateur interagisse uniquement avec le Shell, tandis que les traitements restent pris en charge par les composants internes du système avant que le résultat ne soit affiché dans un second terminal.

Notre objectif est donc de proposer une architecture simple dans laquelle chaque élément possède un rôle précis : saisie des commandes, coordination des traitements, gestion du système de fichiers, stockage et affichage.

2. Architecture envisagée

Nous proposons une architecture composée de cinq composants. Le schéma ci-dessous présente leur rôle ainsi que la manière dont nous envisageons la circulation d'une commande dans le système.

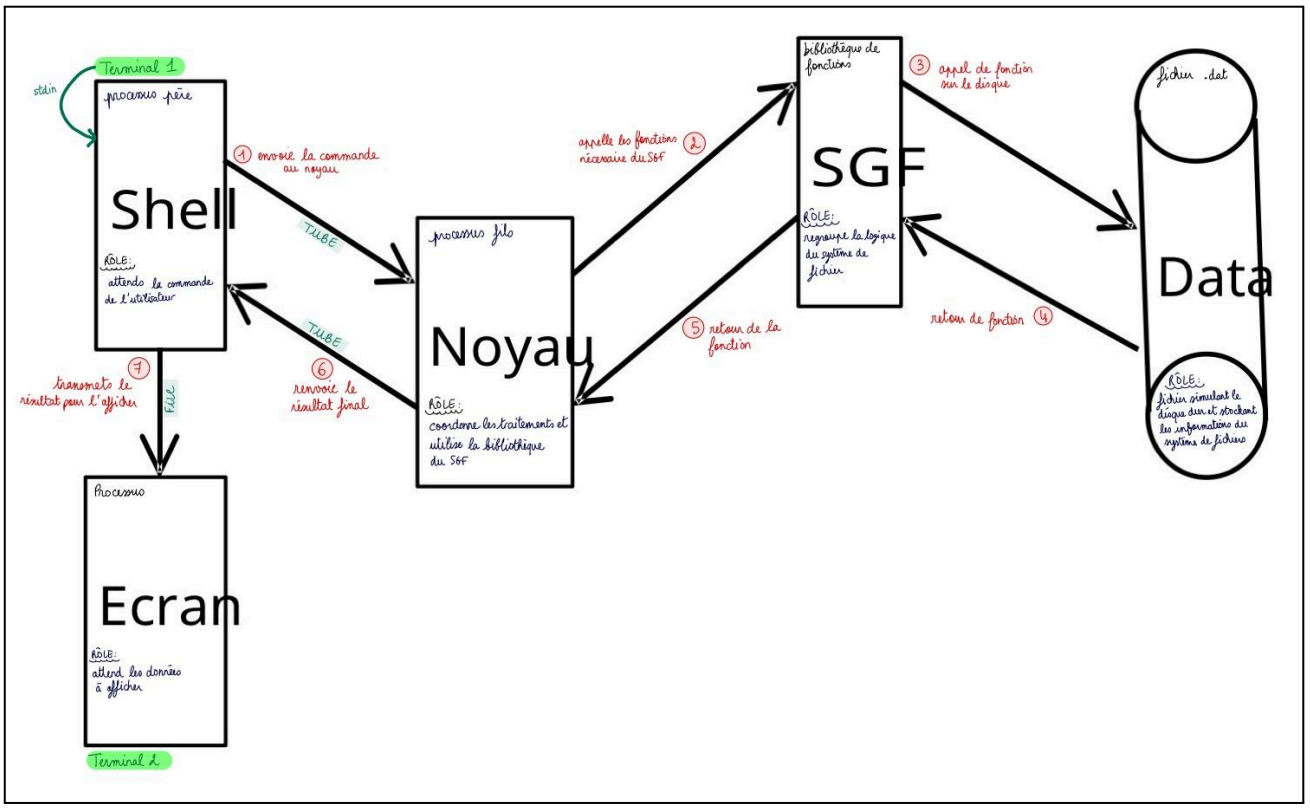


Figure 1 : Architecture générale du système

2.1 Les composants principaux

Le Shell est le processus père du Noyau, exécuté dans le terminal 1. Il attend les commandes de l'utilisateur et constitue le point d'entrée du système.

Le Noyau serait créé comme processus fils du Shell. Son rôle est de coordonner les traitements et de déclencher les opérations nécessaires à partir des commandes reçues.

Pour regrouper la logique du système de fichiers, nous prévoyons un SGF sous la forme d'une bibliothèque de fonctions. Celui-ci prendra en charge les opérations sur les fichiers et les répertoires et accédera au disque virtuel lorsque cela sera nécessaire.

Le stockage serait assuré par un fichier .dat jouant le rôle de disque virtuel. Nous l'utiliserons pour simuler le disque dur et conserver les informations du système de fichiers.

Enfin, nous prévoyons un programme Écran exécuté dans un second terminal. Son rôle serait uniquement d'attendre et d'afficher les résultats qui lui sont transmis.

2.2 Circulation d'une commande et communications

Lorsqu'une commande est saisie, le Shell la transmet au Noyau par un premier tube anonyme.

Le Noyau appelle ensuite la fonction du SGF correspondant à l'opération demandée, si un accès au stockage est nécessaire, cette fonction lit ou modifie le fichier .dat.

Le résultat revient ensuite au Noyau, qui le renvoie au Shell par un second tube anonyme.

Le Shell transmet enfin les données au programme Écran par une file de messages afin qu'elles soient affichées dans le terminal 2.

Ce choix nous permet d'utiliser les tubes pour les échanges entre les processus père et fils, les appels de fonctions entre le Noyau et la bibliothèque SGF, puis une file de messages pour communiquer avec le processus Écran exécuté séparément.

3. Conception du système de fichiers

Nous souhaitons centraliser toute la logique du système de fichiers dans le SGF afin que le Noyau n'ait pas à connaître directement l'organisation interne du stockage. Le fichier .dat servirait de disque virtuel et devra contenir les informations nécessaires aux différentes opérations proposées par le SGF.

Nous envisageons un placement indexé des blocs de données des fichiers sur le disque virtuel afin de pouvoir faire évoluer plus facilement leur organisation et leur taille. Les blocs d'un fichier peuvent ainsi être répartis à différents emplacements et retrouvés à partir des informations contenues dans son inode, ce qui évite de devoir déplacer l'ensemble du fichier lorsqu'une modification intervient. La structure précise de l'inode n'est pas encore définie à ce stade.

Cette séparation nous permet de distinguer clairement la coordination des traitements, assurée par le Noyau, de la gestion et du stockage des données, pris en charge par le SGF et le disque virtuel.

4. Organisation logicielle et interfaces

Nous voulons organiser le développement en conservant les responsabilités définies dans l'architecture. Pour que les composants puissent être développés séparément, nous devrons fixer en commun les interfaces qui les relient : le format des données échangées dans les deux tubes, les fonctions exposées par le SGF et le format des messages envoyés au programme Écran.

L'objectif est que chaque partie puisse évoluer indépendamment sans remettre en cause le fonctionnement général du système.

5. Organisation du travail dans l'équipe

Nous envisageons de répartir le développement autour de trois ensembles : le Shell et les communications, le Noyau et la coordination des traitements, puis le SGF avec le disque virtuel.

Avant cette répartition, nous devrons définir ensemble les interfaces communes afin de garantir la compatibilité entre les différentes parties. Nous pourrons ensuite intégrer progressivement les modules et vérifier le fonctionnement complet d'une commande, depuis sa saisie jusqu'à son affichage dans le second terminal.